

Overcoming Technical Constraints to Privacy-First Architecture

Introduction

Privacy-by-design is a long-standing principle in cybersecurity, yet many modern systems still lack robust privacy safeguards by default. Users and regulators increasingly demand solutions that **empower individuals with control over their data**, but established platforms often fall short. The reasons go beyond business reluctance – there are genuine technical constraints at play. Legacy architectures and **technical debt** built up over years can make it difficult to bake privacy into existing products. The result is an industry paradox: we know how to design privacy-first systems in theory, but converting large-scale services to truly isolate and protect user data remains a challenge. This white paper explores how technical limitations hinder privacy-first solutions, and outlines an architectural approach to overcome these constraints.

The Impact of Technical Debt on Privacy

Many organizations attribute the lack of privacy features to business priorities or profit motives. In reality, **underlying technology choices and legacy codebases have equal blame**. Systems not originally designed for data isolation face steep hurdles retrofitting privacy features. For example, multi-tenant cloud applications often mix data and functionality in ways that are hard to untangle, making per-user encryption or on-premise deployment nearly impossible without a ground-up redesign. This **“privacy tech debt”** means even well-intentioned companies struggle to offer privacy-respecting options. Every new privacy control added on top of a legacy stack can introduce complexity and overhead, sometimes creating new vulnerabilities or usability issues ¹ ². As a result, products may remain cloud-dependent and centralized, *forcing* users to trust providers with sensitive data instead of giving users true data sovereignty.

To break this cycle, organizations must recognize privacy as an **architectural concern** rather than an afterthought. That means evaluating how easily a system can be run in isolated modes (for example, in a single-tenant container for one user or in an offline environment) and treating that capability as a core requirement. Unfortunately, many platforms score poorly on this metric today – their tightly coupled services, proprietary protocols, and monolithic designs simply cannot be flipped into an isolated *“privacy mode.”* Addressing this gap will require refactoring and modernizing core components, essentially paying down the technical debt that prevents privacy features. In short, overcoming the **constraints of legacy technology** is often the first step toward delivering true privacy-by-design solutions.

Isolated Microservices for Privacy by Design

A promising architectural pattern for privacy-first design is the use of **isolated microservice pods** that can operate independently per user or context. Instead of one giant service handling all users, the system is composed of modular services (or microservices) that communicate via well-defined interfaces. Critically, these services can be *deployed in two modes*: (1) a *distributed cloud mode* where each component runs at scale (typical microservice deployment), or (2) a *consolidated mode* where all

required components run together in a single container or virtual machine dedicated to one user or one small group.

This approach has been demonstrated in practice. For example, Dinis Cruz's recent work on a man-in-the-middle proxy solution uses multiple serverless web services (cache service, HTML service, semantic-text service, mitmproxy API service) that can run in concert ³. In a cloud deployment these services communicate over the network, but they can also be **wired together in-memory** within a single process or container for an isolated deployment. This flexibility means the entire product can be packaged to run as a **self-contained unit** – effectively an “*appliance*” that a user or organization controls fully. Cruz's MITM Proxy architecture includes a *Standalone (Air-Gapped) Mode* where the proxy and all supporting services run on a single host without external dependencies. In this mode, a company could deploy the solution in an offline or high-security environment (even on an isolated EC2 instance or a Kubernetes node dedicated to one user's session) without sacrificing functionality. The ability to scale *down* to a contained, per-user instance is the key enabler of privacy here. It ensures that a user's data can be processed locally or in their own cloud account, rather than forcing all data through a multi-tenant SaaS pipeline.

Isolated microservice pods align with the principle of **data minimization**. Each instance only holds the data for its specific user or task, and only for the duration needed. This greatly limits the exposure of personal data. If thousands of users use a service, the old approach might pool all their data in one database or memory space; the new approach keeps each user's information in a separate silo (logical and possibly physical). Designing software this way from the start – where any component can be deployed as an isolated service – requires forethought. Services must be loosely coupled and stateless where possible, making it easier to lift and “transplant” them into a new context. Additionally, deployment automation (like container orchestration) is needed to spin up per-user service pods on demand. Modern DevOps and cloud tools have made this feasible – orchestrators can run hundreds of lightweight containers, one per user, with dynamic scheduling to optimize resource use.

Ephemeral Pods as Digital Vaults

A cornerstone of privacy-first architecture is treating each user's instance as a **digital vault** for that user's data. In the isolated pod model, *all sensitive data and computation for a user occurs within their dedicated container*, which can be heavily sandboxed. We can enforce that any data at rest in the container is encrypted by keys that only the user (or the user's client) possesses. In practice, this means even if the container's storage is compromised or inspected, the data remains unintelligible without the live decryption key. The key can be provided at runtime (for example, by the user's device or a secure key management service) and reside only in volatile memory. When the service pod shuts down, the key is dropped and the data on disk stays encrypted. **Even the service operator cannot read the stored data** if they don't have the user's key. This approach dramatically reduces risk – it's analogous to client-side encryption but applied at the level of a personal microservice instance.

Recent advances in container and cloud technology support this model. Containerization already offers strong isolation between processes, and new techniques like **confidential computing** even allow encryption of memory and CPU state so that data is protected during processing. For instance, some container platforms now support running workloads in enclaves where *the container is encrypted at rest, in transit, and even while running*, meaning no intermediate plain-text data ever touches the disk unencrypted ⁴. In other words, we are approaching a paradigm where **data can remain encrypted in use**, not just at rest and in transit. As a concrete example, hardware-backed features can ensure that not even a cloud provider or host OS can peek into an executing container's memory ⁵. While such

advanced features are still emerging, they complement the privacy-by-design goal: only the user (or their delegate service) holds the decryption capability.

Within an ephemeral user-specific pod, we also gain excellent auditability and control. Since the pod is the sole locus of that user's computation, it can produce a detailed log of every access or operation on the data. One could know exactly which code modules ran and when, which external calls were made, and who initiated them – a clear trail that is much harder to obtain in multi-tenant systems. Moreover, if a user decides to revoke access, they can destroy the pod or revoke the key, immediately rendering any residual data unusable. The **ephemeral nature** of these vaults means that when they're not needed, they can be shut down – leaving no persistent honeypots of personal data. This greatly limits the window of opportunity for attackers or unauthorized access. In essence, each user pod **embodies the principle of least privilege**: it has exactly the data and code needed for that user's purpose, nothing more, and exists only for the time it is required.

Benefits of Minimalist Service Pods

An added benefit of this architecture is the possibility of **minimalist code footprints** for each service instance. Traditional platforms often load a huge codebase to serve all users and all features, even if any given user only needs a small subset of functionality. With a per-task or per-user service, we can tailor the deployed code to the specific task. This is where modern AI assistance (like large language models) might play a role: automatically determining the minimal set of components needed to fulfill a user's request and deploying a pared-down instance (a concept Cruz refers to as finding the "minimal viable pod"). By stripping out unnecessary modules, each pod becomes smaller and simpler, which improves security and performance. Fewer lines of code and fewer features mean fewer potential vulnerabilities and less that can go wrong. This also eases **code auditing and validation**, since one can analyze a self-contained microservice that performs one job, rather than wading through an entire monolithic application.

From a security perspective, many breaches and privacy incidents stem from overly complex systems where a flaw in one part can be exploited to cascade into a major data leak. With minimal isolated pods, the blast radius of any single flaw is drastically reduced – a bug would only affect that one user's instance, not a central database of thousands of users. This fragmentation is aligned with emerging privacy architecture principles that favor *fragmentation over integration and disposability over permanence* ^{6 7}. If each piece is small and ephemeral, it's harder for attackers to exploit systemic weaknesses or to persist in the environment.

Lean service pods are also cost-efficient in the long run. They consume fewer resources per user when correctly sized. And because they can scale horizontally, you can allocate just enough CPU/memory for active users and reclaim it when they are done. This granularity can drive down cloud costs compared to running one massive always-on service. In essence, *privacy-first design can coincide with efficient design*: isolating and minimizing components often yields a lean architecture that avoids the bloat of all-in-one systems.

Economic Feasibility and Scalability

A common concern is whether deploying per-user or per-use-case containers is economically viable. It might sound expensive to give every user their own suite of microservices. However, with today's container orchestration and serverless technologies, this model can actually be highly cost-effective. Containers can be instantiated in seconds and terminated when idle, so cloud infrastructure is used on-demand. If a user isn't actively using their "personal instance," it can scale down to zero consumption.

Meanwhile, multi-tenant architectures typically run 24/7 and must be over-provisioned for peak loads, which can waste resources. Fine-grained isolation lets us apply **true pay-per-use** economics, potentially making privacy-first deployments *cheaper* at scale, not more expensive.

There are, of course, overheads to consider – container orchestration, image management, and per-instance monitoring all introduce complexity. But large-scale systems like Kubernetes have proven capable of managing thousands of pods efficiently in production environments. Cloud providers are also introducing managed services for isolated functions (e.g. per-request containers or micro-VMs) which could further streamline this approach. The key is that privacy-first architecture needs to be **designed with business viability in mind**. It's not enough to be secure and private; the solution must also meet performance and cost requirements to be sustainable. Fortunately, the modular approach can help optimize both: by running only what is needed, where it's needed. For example, an edge deployment could run these pods on customer premises for sensitive processing, while less sensitive parts remain in a shared cloud – a hybrid approach that balances cost and privacy.

Another aspect of feasibility is **user experience and trust**. A privacy-first service will highlight to users that their data is processed in a dedicated environment under their control. This can become a selling point, adding business value in the form of user trust and compliance benefits. Organizations can differentiate themselves by saying, “your data never leaves your container running on your infrastructure” or “each user’s data is compartmentalized in real time.” In sectors with strict data regulations (healthcare, finance, government), this capability is not just nice-to-have but often a requirement. Thus, investing in architecture that enables isolated, encrypted processing pods can open doors to markets and use cases that traditional SaaS cannot serve due to privacy restrictions.

The Need for an Open Ecosystem

To truly realize privacy-first architectures, openness is crucial at multiple levels. **Open-source code** is important so that the community (and customers) can audit the software for privacy and security issues. When code is proprietary, users must blindly trust the vendor’s claims about data handling. In contrast, an open-source privacy solution allows independent verification that no data is surreptitiously leaking or being retained longer than intended. Dinis Cruz argues that making code, data formats, and schemas open is the only way such solutions will scale and gain trust. Openness also prevents **vendor lock-in**, which aligns with privacy goals – users should be able to run their data processing wherever they want (their own cloud or device), not be locked to the vendor’s servers. Using open data schemas and standards ensures that an isolated instance running in a sovereign environment will produce data that is portable and interoperable with other systems, instead of trapping user data in a proprietary silo.

We can draw a parallel with other infrastructure: for instance, organizations choosing an on-premise vector database instead of a cloud API do so for control and privacy reasons, but they benefit only if the software is open and self-hostable ⁸ ⁹. The same applies to any privacy-first component – whether it’s a proxy, an AI model, or a data transformation tool, it should ideally be open and deployable in any environment. This encourages an ecosystem where multiple vendors and community contributors can improve the code, find bugs, and adapt the software to new use cases, all of which accelerates innovation. Security through obscurity is a poor strategy; security (and privacy) through transparency is far more robust.

Finally, an open ecosystem fosters **collaboration and standardization**. If many companies adopt the approach of isolated data pods, having open standards for how these pods authenticate, log events, or hand off data would enable them to interoperate. For example, one could envision a standard for

exporting an encrypted data vault from one system and importing it to another securely, if both adhere to open interface specifications. Such possibilities only emerge when the industry coalesces around open solutions rather than closed, proprietary ones.

Conclusion

Privacy-first solutions are not only an ethical choice but increasingly a competitive differentiator in the cybersecurity and AI landscape. Achieving them requires more than policy changes – it demands **architectural modernization**. By addressing the technical constraints head-on, we can redesign systems to support *privacy and user autonomy from the ground up*. This paper discussed how technical debt and legacy design hamper privacy features, and how adopting isolated, ephemeral microservice pods can break through those limitations. The proposed architecture – essentially per-user or per-task “digital vaults” implemented as containerized services – offers strong isolation, user control over data (through runtime-only keys and encryption), and potential for minimal attack surface. Early implementations of these ideas, such as Cruz’s containerized MITM proxy services, show that it’s feasible to run complex workflows in a self-contained, privacy-preserving manner ³.

To move forward, organizations should start **measuring their systems’ isolatability** – how easily can you deploy your solution in a fully isolated mode for one customer or one dataset? Any hard dependency on central infrastructure is a point of friction that might be removed with refactoring. Reducing those dependencies incrementally pays off in flexibility: it becomes easier to comply with data residency laws, to offer on-premise versions, or to hand control back to users. Furthermore, embracing open-source practices and community standards will accelerate progress by building trust and enabling collaboration on privacy tech. In the end, privacy-first architecture is about *respect*: respecting user’s rights to their data and aligning our technology with that respect. With thoughtful design and modern tools, we can create systems where privacy isn’t an afterthought or a checkbox, but a fundamental property of how they operate. This not only meets regulatory and user expectations, but also leads to resilient, modular systems that are robust in the face of security threats. The path to such architectures may be challenging, but it is clearly one worth pursuing for the future of secure and ethical technology.

References

- Cruz, D. (2026). *Man-in-the-Middle Proxy Service – Deployment Modes*. [Dev Brief – NotebookLM Infographics]. (Demonstrates a microservice-based proxy that can run in isolated container mode for privacy.)
- Cruz, D. (2026). *Standalone (Air-Gapped) MITM Proxy Service – Dev Brief*. [NotebookLM Infographics]. (Outlines an offline deployment of a proxy service as a self-contained user instance.)
- **LinkedIn Post – Dinis Cruz (Jan 2026)**: Description of a MITM proxy UI with multiple serverless services working in concert ³.
- **Apptainer Encrypted Container Documentation**: Example of containers encrypted in use (confidential computing) ⁴ ⁵.
- DisappearMe.AI (2025). *The Architecture of Privacy Vulnerability*. (Discusses how adding privacy layers can introduce new attack surfaces, advocating for fragmented, disposable components) ¹ ².
- Blue Marloc (2024). *Privacy-Preserving On-Premises AI Infrastructure*. (Highlights importance of on-prem isolation and open-source alternatives for data control) ⁸ ⁹.

1 2 6 7 Privacy Architecture Risk Assessment: Systemic Vulnerabilities in Modern Privacy Solutions

<https://disappearme.ai/blog/architecture-privacy-vulnerability-every-privacy-solution-creates-new-attack-surfaces-disappearme-ai>

3 Here is something I just 'vibe coded' (from a detailed 500+ line briefing + reference code + implementation/coding guidance) As the name implies, this UI talks with the Mitmproxy API and simulates... | Dinis Cruz

https://www.linkedin.com/posts/diniscruz_here-is-something-i-just-vibe-coded-from-activity-7419936869531996160-yG5M

4 Encrypted Containers — Apptainer User Guide main documentation

<https://apptainer.org/docs/user/main/encryption.html>

5 Container Security & Encryption - Docker Community Forums

<https://forums.docker.com/t/container-security-encryption/137699>

8 9 Blog

<https://bluemarloc.com/index.php/privacy-preserving-premises-ai-agent-infrastructure>